

Linux-Kernel Memory Model

ISO/IEC JTC1 SC22 WG21 P0124R3 - 2017-06-26

Paul E. McKenney, paulmck@linux.vnet.ibm.com

Ulrich Weigand, Ulrich.Weigand@de.ibm.com

Andrea Parri, parri.andrea@gmail.com

Boqun Feng (Intel), boqun.feng@gmail.com

History

This is a revision of [N4444](#), updated to add Linux-kernel architecture advice and add more commentary on optimizations. This is a revision of [N4374](#), updated to cover the new `READ_ONCE()` and `WRITE_ONCE()` API members. N4374 was itself a revision of [N4322](#), with updates based on subsequent discussions. This revision adds references to litmus tests in userspace RCU, a paragraph stating goals, and a section discussing the relationship between volatile atomics and loop unrolling.

Introduction

The Linux-kernel memory model is currently defined very informally in the [memory-barriers.txt](#) and [atomic_ops.txt](#) files in the source tree. Although these two files appear to have been reasonably effective at helping kernel hackers understand what is and is not permitted, they are not necessarily sufficient for deriving the corresponding formal model. This document is a first attempt to bridge this gap. A candidate formal model is described in the two-part LWN series [here](#) and [here](#), which is in turn an elaboration of the model described [here](#) ([video](#), [extended presentation](#), [litmus-test repository](#)).

The hope is that this document will help the C and C++ standard committees understand the existing practice and the constraints from the Linux kernel, and also that it will help the Linux community evaluate which portions of the C11 and C++11 memory models might be useful in the Linux kernel.

All that said, the Linux kernel does not mark declarations of variables and structure fields that are to be manipulated atomically. This means that the `__atomic` built-ins, which in gcc generate the same code as their C11-atomic counterparts are easier to apply to the Linux kernel than are the C11 atomics.

1. [Variable Access](#)
2. [Memory Barriers](#)
3. [Locking Operations](#)
4. [Atomic Operations](#)
5. [Control Dependencies](#)
6. [RCU Grace-Period Relationships](#)
7. [Summary of Differences With Examples](#)
8. [So You Want Your Arch To Use C11 Atomics...](#)
9. [Summary](#)

Variable Access

Loads from and stores to shared (but non-atomic) variables should be protected with the `READ_ONCE()`, `WRITE_ONCE()`, and [ACCESS_ONCE\(\)](#) macros, for example:

```
r1 = READ_ONCE(x);
WRITE_ONCE(y, 1);
r2 = ACCESS_ONCE(x);
ACCESS_ONCE(y) = 1;
```

A `READ_ONCE()`, `WRITE_ONCE()`, and `ACCESS_ONCE()` accesses may be modeled as a `volatile memory_order_relaxed` access. However, please note that these macros are defined to work properly only for properly aligned machine-word-sized variables. Applying `ACCESS_ONCE()` to a large array or structure is unlikely to do anything useful, and use of `READ_ONCE()` and `WRITE_ONCE()` in this situation can result in load-tearing and store-tearing. Nevertheless, this is their definition. Linux-kernel developer would most certainly not be thankful for the compiler to add locks to `READ_ONCE()`, `WRITE_ONCE()`, and `ACCESS_ONCE()` when applied to oversized objects.

Note that the `volatile` is absolutely required: Non-volatile `memory_order_relaxed` is not sufficient. To see this, consider that `READ_ONCE()` can be used to prevent concurrently modified accesses from being hoisted out of a loop or out of unrolled instances of a loop. For example, given this loop:

```
while (tmp = atomic_load_explicit(a, memory_order_relaxed))
    do_something_with(tmp);
```

The compiler would be permitted to unroll it as follows:

```
while (tmp = atomic_load_explicit(a, memory_order_relaxed))
    do_something_with(tmp);
    do_something_with(tmp);
    do_something_with(tmp);
    do_something_with(tmp);
}
```

This would be unacceptable for real-time applications, which need the value to be reloaded from `a` on each iteration, unrolled or not. The `volatile` qualifier prevents this transformation. For example, consider the following loop:

```
while (tmp = READ_ONCE(a))
    do_something_with(tmp);
```

This loop could still be unrolled, but the read would also need to be unrolled, for example, like this:

```
for (;;) {
    if (!(tmp = READ_ONCE(a)))
        break;
    do_something_with(tmp);
    if (!(tmp = READ_ONCE(a)))
        break;
    do_something_with(tmp);
    if (!(tmp = READ_ONCE(a)))
        break;
    do_something_with(tmp);
    if (!(tmp = READ_ONCE(a)))
        break;
    do_something_with(tmp);
}
```

Note that use of the new `READ_ONCE()` and `WRITE_ONCE()` macros are recommended for new code, in fact, `ACCESS_ONCE()` might well be phased out. Of course, one advantage of such a phase-out is that `READ_ONCE()` and `WRITE_ONCE()` are a better match for the C and C++ `memory_order_relaxed` loads and stores, give or take volatility.

At one time, `gcc` guaranteed that properly aligned accesses to machine-word-sized variables would be atomic. Although `gcc` no longer documents this guarantee, there is still code in the Linux kernel that relies on it. These accesses could be modeled as non-volatile `memory_order_relaxed` accesses.

The Linux kernel provides `atomic_t` and `atomic_long_t` types. These have `atomic_read()` and `atomic_long_read()` operations that provide non-RMW loads from the underlying variable. They also have `atomic_set()` and `atomic_long_set()` operations that provide non-RMW stores into the the underlying variable. These were originally intended for single-threaded initialization-time and cleanup-time accesses to atomic variables, however, they have since been adapted to operate in a manner similar to `memory_order_relaxed` loads and stores.

The `atomic_t` and `atomic_long_t` types have quite a few other operations that are described in the “Atomic Operations” section. These types could potentially be modeled as volatile atomic `int` for `atomic_t` and volatile atomic `long` for `atomic_long_t`, however, anyone using such a strategy could expect great scrutiny of the code generated at initialization time, when there is no possibility of concurrent access. (Implementations that implement aligned machine-word-sized relaxed atomic loads and stores as normal load and store instructions will pass scrutiny, at least assuming that pointers are machine-word-sized.)

An `smp_store_release()` may be modeled as a volatile `memory_order_release` store. Similarly, an `smp_load_acquire()` may be modeled as a volatile `memory_order_acquire` load.

```
r1 = smp_load_acquire(x);
smp_store_release(y, 1);
```

Members of the `rcu_dereference()` family can be modeled as `memory_order_consume` loads. Members of this family include: `rcu_dereference()`, `rcu_dereference_bh()`, `rcu_dereference_sched()`, `srcu_dereference()`, and `lockless_dereference()`. However, `rcu_dereference()` should be representative for litmus-test purposes, at least initially. Similarly, `rcu_assign_pointer()` can be modeled as a `memory_order_release` store. (That said, if `rcu_assign_pointer()` is storing a `NULL` or a pointer to constant data, for example, compile-time initialized data, then `rcu_assign_pointer()` may be modeled as a `memory_order_relaxed` store.

The `set_mb()` function assigns the specified value to the specified variable, then executes a full memory barrier, which is described in the next section. This isn't as strong as a `memory_order_seq_cst` store because the following code fragment does not guarantee that the stores to `x` and `y` will be ordered.

```
smp_store_release(x, 1);
```

```
set_mb(y, 1);
```

That said, `set_mb()` provides exactly the ordering required for manipulating task state, which is the job for which it was created.

Memory Barriers

The Linux kernel has a variety of memory barriers:

1. `barrier()`, which can be modeled as an `atomic_signal_fence(memory_order_acq_rel)` or an `atomic_signal_fence(memory_order_seq_cst)`.
2. `smp_mb()`, which does not have a direct C11 or C++11 counterpart. On an ARM, PowerPC, or x86 system, it can be modeled as a full memory-barrier instruction (`dmb`, `sync`, and `mfence`, respectively). On an Itanium system, it can be modeled as an `mf` instruction, but this relies on `gcc` emitting an `ld.acq` for an `ACCESS_ONCE()` load and an `st.rel` for an `ACCESS_ONCE()` store. (Peter Zijlstra of Intel notes that although IA64's reference manual claims instructions with acquire and release semantics, the actual hardware implements only full barriers. See commit `e4f9bfb3feae` ("ia64: Fix up `smp_mb_{before,after}_clear_bit()`") for Linux-kernel changes based on this situation. Tony Luck and Fenghua Yu are the IA64 maintainers for the Linux kernel.)
3. `smp_rmb()`, which can be modeled (overly conservatively) as an `atomic_thread_fence(memory_order_acq_rel)`. One difference is that `smp_rmb()` need not order prior loads against later stores, or prior stores against later stores. Another difference is that `smp_rmb()` need not provide any sort of transitivity, having (lack of) transitivity properties similar to ARM's or PowerPC's address/control/data dependencies.
4. `smp_wmb()`, which can be modeled (again overly conservatively) as an `atomic_thread_fence(memory_order_acq_rel)`. One difference is that `smp_wmb()` need not order prior loads against later stores, nor prior loads against later loads. Similar to `smp_rmb()`, `smp_wmb()` need not provide any sort of transitivity.
5. `smp_read_barrier_depends()`, which is a no-op on all architectures other than Alpha. On Alpha, `smp_read_barrier_depends()` may be modeled as a `atomic_thread_fence(memory_order_acq_rel)` or as a `atomic_thread_fence(memory_order_seq_cst)`.
6. `smp_mb__before_atomic()`, which provides a full memory barrier before the immediately following non-value-returning atomic operation.
7. `smp_mb__after_atomic()`, which provides a full memory barrier after the immediately preceding non-value-returning atomic operation. Both `smp_mb__before_atomic()` and `smp_mb__after_atomic()` are described in more detail in the later section on atomic operations.
8. `smp_mb__after_unlock_lock()`, which provides a full memory barrier after the immediately preceding lock operation, but only when paired with a preceding unlock operation by this same thread or a preceding unlock operation on the same lock variable. The use of `smp_mb__after_unlock_lock()` is described in more detail in the section on locking.

There are some additional memory barriers including `mmiowb()`, however, these cover interactions with memory-mapped I/O, so have no counterpart in C11 and C++11 (which is most likely as it should be for the foreseeable future).

Some use cases for these memory barriers may be found [here](#). These are for the userspace RCU library, so drop the leading `cmm_` to get the corresponding Linux-kernel primitive. For example, the userspace `cmm_smp_mb()` primitive translates to the Linux-kernel `smp_mb()` primitive.

Locking Operations

The Linux kernel features “roach motel” ordering on its locking primitives: Prior operations can be reordered to follow a later acquire, and subsequent operations can be reordered to precede an earlier release. The CPU is permitted to reorder acquire and release operations in this way, but the compiler is not, as compiler-based reordering could result in deadlock.

Note that a release-acquire pair does not necessarily result in a full barrier. To see this consider the following litmus test, with `x` and `y` both initially zero, and locks `l1` and `l3` both initially held by the threads releasing them:

```
Thread 1                      Thread 2
-----                      -----
y = 1;                        x = 1;
spin_unlock(&l1);              spin_unlock(&l3);
spin_lock(&l2);                spin_lock(&l4);
r1 = x;                       r2 = y;

assert(r1 != 0 || r2 != 0);
```

In the above litmus test, the assertion can trigger, meaning that an unlock followed by a lock is not guaranteed to be a full memory barrier. And this is where `smp_mb__after_unlock_lock()` comes in:

```
Thread 1                      Thread 2
-----                      -----
y = 1;                        x = 1;
```

```

spin_unlock(&l1);          spin_unlock(&l3);
spin_lock(&l2);            spin_lock(&l4);
smp_mb__after_unlock_lock(); smp_mb__after_unlock_lock();
r1 = x;                    r2 = y;

assert(r1 != 0 || r2 != 0);

```

In contrast, after addition of `smp_mb__after_unlock_lock()`, the assertion cannot trigger.

The above example showed how `smp_mb__after_unlock_lock()` can cause an unlock-lock sequence in the same thread to act as a full barrier, but it also applies in cases where one thread unlocks and another thread locks the same lock, as shown below:

Thread 1	Thread 2	Thread 3
-----	-----	-----
y = 1;	spin_lock(&l1);	x = 1;
spin_unlock(&l1);	smp_mb__after_unlock_lock();	smp_mb();
	r1 = y;	r3 = y;
	r2 = x;	
assert(r1 == 0 r2 != 0 r3 != 0);		

Without the `smp_mb__after_unlock_lock()`, the above assertion can trigger, and with it, it cannot. The fact that it can trigger without might seem strange at first glance, but locks are only guaranteed to give sequentially consistent ordering to their critical sections. If you want an observer thread to see the ordering without holding the lock, you need `smp_mb__after_unlock_lock()`. (Note that there is some possibility that the Linux kernel's memory model will change such that an unlock followed by a lock forms a full memory barrier even without the `smp_mb__after_unlock_lock()`.)

The Linux kernel has an embarrassingly large number of locking primitives, but `spin_lock()` and `spin_unlock()` should be representative for litmus-test purposes, at least initially.

Interestingly enough, the Linux kernel's locking operations can be argued to be weaker than those of C11. This argument is based on interpretation of 29.3p3 of the C++11 standard, which states in a non-normative note:

Although it is not explicitly required that S include locks, it can always be extended to an order that does include lock and unlock operations, since the ordering between those is already included in the “happens before” ordering.

That said, [current C11 formal memory models](#) specify locking primitives to have roughly the same strength as those of the Linux kernel, based on the following two litmus tests:

```

1 int main()
2 {
3     atomic_int x = 0;
4     atomic_int y = 0;
5     mutex m1;
6     mutex m2;
7     mutex m3;
8     mutex m4;
9
10    {{{ { m1.lock();
11          y.store(1, memory_order_relaxed);
12          m1.unlock();
13          m2.lock();
14          r1 = x.load(memory_order_relaxed);
15          m2.unlock(); }
16
17    ||| { m3.lock();
18          x.store(1, memory_order_relaxed);
19          m3.unlock();
20          m4.lock();
21          r2 = y.load(memory_order_relaxed);
22          m4.unlock(); }
23    }}};
24
25    return 0;
26 }

```

This first litmus test can observe both `r1` and `r2` equal to zero, which would be prohibited if locking primitives enforced SC behavior.

The second litmus test substitutes a fence for one of the unlock-lock pairs:

```

int main()
1 {
2     atomic_int x = 0;
3     atomic_int y = 0;

```

```

4     mutex mtx;
5
6     {{{ { mtx.lock();
7           x.store(1, memory_order_relaxed);
8           mtx.unlock();
9           mtx.lock();
10          r0 = y.load(memory_order_relaxed);
11          mtx.unlock(); }
12
13     ||| { y.store(1, memory_order_relaxed);
14           atomic_thread_fence(memory_order_seq_cst);
15           r1 = x.load(memory_order_relaxed); }
16     }}};
17
18     return 0;
19 }

```

This second litmus test can also observe both `r1` and `r2` equal to zero, which again would be prohibited if locking primitives enforced SC behavior.

It is quite possible that the Linux kernel's locking primitives will be strengthened so that an unlock-lock pair implies a full memory barrier. The motivation is a pair of obscure Linux-kernel primitives named `spin_unlock_wait()` and `queued_spin_unlock_wait()`, which are guaranteed to wait until the current holder of the lock (if any) releases it. This primitive provides RCU-like semantics, where the holders of the lock are considered readers. The ordering semantics are a bit tricky.

If this lock strengthening happens, the shoe will be on the other foot, so that an alternative interpretation of 29.3p3 would provide weaker C11 locking primitives.

Atomic Operations

Atomic operations have three sets of operations, those that are defined on `atomic_t`, those that are defined on `atomic_long_t`, and those that are defined on aligned machine-sized variables, currently restricted to `int` and `long`. However, in the near term, it should be acceptable to focus on a small subset of these operations.

Variables of type `atomic_t` may be stored to using `atomic_set()` and variables of type `atomic_long_t` may be stored to using `atomic_long_set()`. Similarly, variables of these types may be loaded from using `atomic_read()` and `atomic_long_read()`. The historical definition of these primitives has lacked any sort of concurrency-safe semantics, so the user is responsible for ensuring that these primitives are not used concurrently in a conflicting manner.

That said, many architectures treat `atomic_read()` `atomic_long_read()` as volatile `memory_order_relaxed` loads and a few architectures treat `atomic_set()` and `atomic_long_set()` as `memory_order_relaxed` stores. There is therefore some chance that concurrent conflicting accesses will be allowed at some point in the future, at which point their semantics will be those of volatile `memory_order_relaxed` accesses. However, as noted earlier, any attempt to implement `atomic_t` and `atomic_long_t` as volatile atomic `int` and `long` can expect great scrutiny of the code generated in cases such as initialization where no concurrent accesses are possible.

The remaining atomic operations are divided into those that return a value and those that do not. The atomic operations that do not return a value are similar to C11 atomic `memory_order_relaxed` operations. However, the Linux-kernel atomic operations that do return a value cannot be implemented in terms of the C11 atomic operations. These operations can instead be modeled as `memory_order_relaxed` operations that are both preceded and followed by the Linux-kernel `smp_mb()` full memory barrier, which is implemented using the `DMB` instruction on ARM and the `sync` instruction on PowerPC. Alternatively, if appropriate, `smp_mb__before_atomic()` and `smp_mb__after_atomic()` could be used in place of `smp_mb()`. Note that in the case of the CAS operations `atomic_cmpxchg()`, `atomic_long_cmpxchg()`, and `cmpxchg()`, the full barriers are required only in the success case as v4.3 (before that, full barriers were required in both cases). Strong memory ordering can be added to the non-value-returning atomic operations using `smp_mb__before_atomic()` before and/or `smp_mb__after_atomic()` after.

For some of the value-returning atomic operations, there are also sets of variants introduced in v4.4. These variants use suffixes to indicate their ordering guarantees. There are three kinds of variants: `_relaxed`, `_acquire` and `_release`, and they are similar to the corresponding C11 `memory_order_relaxed`, `memory_order_acquire` and `memory_order_release` atomic operations, except that they are volatile, which means they won't be optimized out or merged with other atomic operations. Note that in the case of the variants of CAS operations `atomic_cmpxchg()`, `atomic_long_cmpxchg()`, and `cmpxchg()`, the ordering guarantees are required only in the success case.

Note that C11 compilers are within their rights to assume data-race freedom when determining what optimizations to carry out. This will break the still-common Linux-kernel practice of assuming relaxed semantics for normal accesses to non-atomic variables, hence the suggestions to disable code-motion optimizations across atomics using full barriers and/or Linux-kernel `barrier()` macros.

The operations are summarized in the following table. An initial implementation of a tool could start with `atomic_add()`, `atomic_sub()`, `atomic_xchg()`, and `atomic_cmpxchg()`.

Operation Class	int	long
Add/Subtract	<code>void atomic_add(int i, atomic_t *v)</code> <code>void atomic_sub(int i, atomic_t *v)</code> <code>void atomic_inc(atomic_t *v)</code> <code>void atomic_dec(atomic_t *v)</code>	<code>void atomic_long_add(long i, atomic_long_t *v)</code> <code>void atomic_long_sub(long i, atomic_long_t *v)</code> <code>void atomic_long_inc(atomic_long_t *v)</code> <code>void atomic_long_dec(atomic_long_t *v)</code>
Add/Subtract, Value Returning, (Variants Available)	<code>int atomic_inc_return(atomic_t *v)</code> <code>int atomic_dec_return(atomic_t *v)</code> <code>int atomic_add_return(int i, atomic_t *v)</code> <code>int atomic_sub_return(int i, atomic_t *v)</code>	<code>long atomic_long_inc_return(atomic_long_t *v)</code> <code>long atomic_long_dec_return(atomic_long_t *v)</code> <code>long atomic_long_add_return(long i, atomic_long_t *v)</code> <code>long atomic_long_sub_return(long i, atomic_long_t *v)</code>
Add/Subtract, Value Returning, (No Variants)	<code>int atomic_inc_and_test(atomic_t *v)</code> <code>int atomic_dec_and_test(atomic_t *v)</code> <code>int atomic_sub_and_test(int i, atomic_t *v)</code> <code>int atomic_add_negative(int i, atomic_t *v)</code>	<code>long atomic_long_inc_and_test(atomic_long_t *v)</code> <code>long atomic_long_dec_and_test(atomic_long_t *v)</code> <code>long atomic_long_sub_and_test(long i, atomic_long_t *v)</code> <code>long atomic_long_add_negative(long i, atomic_long_t *v)</code>
Exchange, (Variants Available)	<code>int atomic_xchg(atomic_t *v, int new)</code> <code>int atomic_cmpxchg(atomic_t *v, int old, int new)</code>	<code>long atomic_long_xchg(atomic_long_t *v, long new)</code> <code>long atomic_long_cmpxchg(atomic_code_t *v, long old, long new)</code>
Conditional Add/Subtract	<code>int atomic_add_unless(atomic_t *v, int a, int u)</code> <code>int atomic_inc_not_zero(atomic_t *v)</code>	<code>long atomic_long_add_unless(atomic_long_t *v, long a, long u)</code> <code>long atomic_long_inc_not_zero(atomic_long_t *v)</code>
Bit Test/Set/Clear (Generic)	<code>void set_bit(unsigned long nr, volatile unsigned long *addr)</code> <code>void clear_bit(unsigned long nr, volatile unsigned long *addr)</code> <code>void change_bit(unsigned long nr, volatile unsigned long *addr)</code>	
Bit Test/Set/Clear, Value Returning (Generic, No Variants)	<code>int test_and_set_bit(unsigned long nr, volatile unsigned long *addr)</code> <code>int _atomic_dec_and_lock(atomic_t *atomic, spinlock_t *lock)</code> <code>int test_and_clear_bit(unsigned long nr, volatile unsigned long *addr)</code> <code>int test_and_change_bit(unsigned long nr, volatile unsigned long *addr)</code>	
Lock-Barrier Operations (Generic)	<code>int test_and_set_bit_lock(unsigned long nr, unsigned long *addr)</code> <code>void clear_bit_unlock(unsigned long nr, unsigned long *addr)</code> <code>void __clear_bit_unlock(unsigned long nr, unsigned long *addr)</code>	
Exchange (Generic, Variants Available)	<code>T *xchg(T *v, new)</code> <code>T *cmpxchg(T *v, T old, T new)</code>	

The rows marked “(Generic)” are type-generic, applying to any aligned machine-word-sized quantity supported by all architectures that the Linux kernel runs on. The set of types is currently those of size `int` and those of size `long`. The “Lock-Barrier Operations” have `memory_order_acquire` semantics for `test_and_set_bit_lock()` and `_atomic_dec_and_lock()`, and have `memory_order_release` for the other primitives. Otherwise, the usual Linux-kernel rule holds: If no value is returned, `memory_order_relaxed` semantics apply, otherwise the operations behave as if there was `smp_mb()` before and after. And for those value-returning primitives, the rows marked “(Variants Available)” have `_relaxed/_acquire/_release` variants, whereas the rows marked “(No Variants)” don’t.

The following table gives rough C11 counterparts for the Linux-kernel atomic operations called out above:

Linux-Kernel Operation	C11 Counterpart
Add/Subtract	
<code>void atomic_add(int i, atomic_t *v)</code> <code>void atomic_long_add(long i, atomic_long_t *v)</code>	<code>atomic_fetch_add_explicit(v, i, memory_order_relaxed)</code>
<code>void atomic_sub(int i, atomic_t *v)</code> <code>void atomic_long_sub(long i, atomic_long_t *v)</code>	<code>atomic_fetch_sub_explicit(v, i, memory_order_relaxed)</code>

void atomic_inc(atomic_t *v) void atomic_long_inc(atomic_long_t *v)	atomic_fetch_add_explicit(v, 1, memory_order_relaxed)
void atomic_dec(atomic_t *v) void atomic_long_dec(atomic_long_t *v)	atomic_fetch_sub_explicit(v, 1, memory_order_relaxed)
Add/Subtract, Value Returning (Variants Available)	
int atomic_inc_return(atomic_t *v) long atomic_long_inc_return(atomic_long_t *v)	atomic_fetch_add(v, 1) - 1
int atomic_dec_return(atomic_t *v) long atomic_long_dec_return(atomic_long_t *v)	atomic_fetch_sub(v, 1) + 1
int atomic_add_return(int i, atomic_t *v) long atomic_long_add_return(long i, atomic_long_t *v)	atomic_fetch_add(v, i) - i
int atomic_sub_return(int i, atomic_t *v) long atomic_long_sub_return(long i, atomic_long_t *v)	atomic_fetch_sub(v, i) + i
Value Returning, No Variants	
int atomic_inc_and_test(atomic_t *v) long atomic_long_inc_and_test(atomic_long_t *v)	atomic_fetch_add(v, 1) == -1
int atomic_dec_and_test(atomic_t *v) long atomic_long_dec_and_test(atomic_long_t *v)	atomic_fetch_sub(v, 1) == 1
int atomic_sub_and_test(int i, atomic_t *v) long atomic_long_sub_and_test(long i, atomic_long_t *v)	atomic_fetch_sub(v, i) == i
int atomic_add_negative(int i, atomic_t *v) long atomic_long_add_negative(long i, atomic_long_t *v)	atomic_fetch_add(v, i) < 0
Exchange, Variants Available	
int atomic_xchg(atomic_t *v, int new) long atomic_long_xchg(atomic_long_t *v, long new) T *xchg(T *v, new)	atomic_exchange(v, new)
int atomic_cmpxchg(atomic_t *v, int old, int new) long atomic_long_cmpxchg(atomic_code_t *v, long old, long new) T *cmpxchg(T *v, T old, T new)	t = old; atomic_compare_exchange_strong_explicit(v, &t, new, memory_order_seq_cst, memory_order_relaxed) return t;
Conditional Add/Subtract	
int atomic_add_unless(atomic_t *v, int a, int u) long atomic_long_add_unless(atomic_long_t *v, long a, long u)	t = atomic_load_explicit(v, memory_order_relaxed); do { if (t == u) return false; } while (!atomic_compare_exchange_weak_explicit(a, t, t + a, memory_order_seq_cst, memory_order_relaxed)); return true;
int atomic_inc_not_zero(atomic_t *v) long atomic_long_inc_not_zero(atomic_long_t *v)	t = atomic_load_explicit(v, memory_order_relaxed); do { if (t == 0) return false; } while (!atomic_compare_exchange_weak_explicit(a, t, t + 1, memory_order_seq_cst, memory_order_relaxed)); return true;
Bit Test/Set/Clear/Change	
	p and mask in below:

	<pre> unsigned long *p = ((unsigned long *)addr) + nr / (sizeof(unsigned long) * CHAR_BIT); unsigned long mask = 1u << (nr % (sizeof(unsigned long) * CHAR_BIT)) </pre>
void set_bit(unsigned long nr, volatile void *addr)	atomic_fetch_or_explicit(p, mask, memory_order_relaxed);
void clear_bit(unsigned long nr, volatile void *addr)	atomic_fetch_and_explicit(p, ~mask, memory_order_relaxed);
void change_bit(unsigned long nr, volatile void *addr)	atomic_fetch_xor_explicit(p, mask, memory_order_relaxed);
int test_and_set_bit(unsigned long nr, volatile void *addr)	return !(atomic_fetch_or_explicit(p, mask, memory_order_relaxed) & mask);
int test_and_clear_bit(unsigned long nr, volatile void *addr)	return !(atomic_fetch_and_explicit(p, ~mask, memory_order_relaxed) & mask);
int test_and_change_bit(unsigned long nr, volatile void *addr)	return !(atomic_fetch_xor_explicit(p, mask, memory_order_relaxed) & mask);
Lock Bit Test and Set/Clear	
	p and mask in below: <pre> unsigned long *p = ((unsigned long *)addr) + nr / (sizeof(unsigned long) * CHAR_BIT); unsigned long mask = 1u << (nr % (sizeof(unsigned long) * CHAR_BIT)) </pre>
int test_and_set_bit_lock(unsigned long nr, volatile void *addr)	return !(atomic_fetch_or_explicit(p, mask, memory_order_acquire) & mask);
void clear_bit_unlock(unsigned long nr, volatile void *addr)	atomic_fetch_and_explicit(p, ~mask, memory_order_release);

The bit test/set/clear and lock-barrier operations map to atomic bit operations, but accept very large bit numbers. The upper bits of the bit number select the unsigned long element of an array, and the lower bits select the bit to operate on within the selected unsigned long.

Control Dependencies

The Linux kernel provides a limited notion of control dependencies, ordering prior loads against control-dependent stores in some cases. Extreme care is required to avoid control-dependency-destroying compiler optimizations. The restrictions applying to control dependencies include the following:

- Control dependencies can order prior loads against later dependent stores, however, they do *not* order prior loads against later dependent loads. (Use `memory_order_consume` OR `memory_order_acquire` if you require this behavior.
- A load heading up a control dependency must use `READ_ONCE()`. Similarly, the store at the other end of a control dependency must also use `WRITE_ONCE()`.
- If both legs of a given `if` or `switch` statement store the same value to the same variable, then those stores cannot participate in control-dependency ordering.
- Control dependencies require at least one run-time conditional that depends on the prior load and that precedes the following store.
- The compiler must perceive both the variable loaded from and the variable stored to as being shared variables. For example, the compiler will not perceive an on-stack variable as being shared unless its address has been taken and exported to some other thread (or alias analysis has otherwise been defeated).
- Control dependencies are not transitive. In this regard, their behavior is similar to ARM or PowerPC control dependencies.

The C and C++ standards do not guarantee any sort of control dependency. Therefore, this list of restriction is subject to change as compilers become increasingly clever and aggressive.

RCU Grace-Period Relationships

The publish-subscribe portions of RCU are captured by the combination of `rcu_assign_pointer()`, which can be modeled as a `memory_order_release` store, and of the `rcu_dereference()` family of primitives, which can be modeled as `memory_order_consume` loads, as was noted earlier.

Grace periods can be modeled as described in Appendix D of [User-Level Implementations of Read-Copy Update](#). There are a

number of grace-period primitives in the Linux kernel, but `rcu_read_lock()`, `rcu_read_unlock()`, and `synchronize_rcu()` are good places to start. The grace-period relationships can be describe using the following abstract litmus test:

Thread 1	Thread 2
-----	-----
<code>rcu_read_lock();</code>	<code>S2a;</code>
<code>S1a;</code>	<code>synchronize_rcu();</code>
<code>S1b;</code>	<code>S2b;</code>
<code>rcu_read_unlock();</code>	

If either of `S1a` or `S1b` precedes `S2a`, then both must precede `S2b`. Conversely, if either of `S1a` or `S1b` follows `S2b`, then both must follow `S2a`. Additional litmus tests may be found [here](#). Again, these are for the userspace RCU library, so drop the leading `cmm_` to get the corresponding Linux-kernel primitives.

Given a high-quality implementation of `memory_order_consume`, RCU can be implemented as a library.

Summary of Differences With Examples

This section looks in more detail at functionality that the Linux kernel provides that is not available from the C11 standard.

1. [ACCESS_ONCE\(\)](#)
2. [smp_mb\(\)](#)
3. [smp_read_barrier_depends\(\)](#)
4. [Locking Operations](#)
5. [Value-Returning Atomics](#)
6. [Control Dependencies](#)

ACCESS_ONCE()

There is no C11 syntax corresponding to `ACCESS_ONCE()`, which enables both loads and stores. However, it appears that `ACCESS_ONCE()` is being replaced by `READ_ONCE()` and `WRITE_ONCE()`, so this situation is likely to be temporary.

smp_mb()

Quoting 29.3p8 of the C++11 standard:

Fences cannot, in general, be used to restore sequential consistency for atomic operations with weaker ordering specifications.

In contrast, `smp_mb()` guarantees to restore sequential consistency among accesses that use `ACCESS_ONCE()`, `READ_ONCE`, `WRITE_ONCE()`, or stronger. For example, the following Linux-kernel code would forbid non-SC outcomes:

```
1 int x, y, r0, r1, r2, r3;
2
3 void thread0(void)
4 {
5     WRITE_ONCE(x, 1);
6 }
7
8 void thread1(void)
9 {
10    WRITE_ONCE(y, 1);
11 }
12
13 void thread2(void)
14 {
15     r0 = READ_ONCE(x);
16     smp_mb()
17     r1 = READ_ONCE(y);
18 }
19
20 void thread3(void)
21 {
22     r2 = READ_ONCE(y);
23     smp_mb()
24     r3 = READ_ONCE(x);
25 }
```

In contrast, the closest C11 analog can permit the non-SC outcomes and still conform to the standard:

```
1 atomic_int x, y;
2 int r0, r1, r2, r3;
```

```

3
4 void thread0(void)
5 {
6     atomic_store_explicit(x, 1, memory_order_relaxed);
7 }
8
9 void thread1(void)
10 {
11     atomic_store_explicit(y, 1, memory_order_relaxed);
12 }
13
14 void thread2(void)
15 {
16     r0 = atomic_load_explicit(x, memory_order_relaxed);
17     atomic_thread_fence(memory_order_seq_cst);
18     r1 = atomic_load_explicit(y, memory_order_relaxed);
19 }
20
21 void thread3(void)
22 {
23     r2 = atomic_load_explicit(y, memory_order_relaxed);
24     atomic_thread_fence(memory_order_seq_cst);
25     r3 = atomic_load_explicit(x, memory_order_relaxed);
26 }

```

That said, it is not clear that anything in the Linux kernel cares whether or not sequential consistency is restored.

smp_read_barrier_depends()

Although it is legal C11 to say “`atomic_thread_fence(memory_order_consume)`”, this is promoted to an acquire fence. Within the Linux kernel, this would have the undesirable effect of promoting `rcu_dereference()` to acquire as well. Linux therefore needs to continue defining `smp_read_barrier_depends()` as `smp_mb()` on DEC Alpha and nothingness elsewhere.

All known C11 implementations currently promote `memory_order_consume` to `memory_order_acquire`, though it is hoped that this situation will improve in the near future.

Locking Operations

Consider the following litmus test:

```

1 void thread0(void)
2 {
3     spin_lock(&my_lock);
4     WRITE_ONCE(x, 1);
5     spin_unlock(&my_lock);
6     spin_lock(&my_lock);
7     r0 = READ_ONCE(y);
8     spin_unlock(&my_lock);
9 }
10
11 void thread1(void)
12 {
13     WRITE_ONCE(y, 1);
14     smp_mb();
15     r1 = READ_ONCE(x);
16 }

```

The Linux kernel is currently within its rights to arrive at the non-SC outcome `r0 == 0 && r1 == 0`. This might change in the near future. One or the other of these states will be inconsistent with C11.

Value-Returning Atomics

Linux's value-returning atomics provide unconditional ordering. For example, in the following code fragment, the outcome `r0 == 1 && r1 == 0` is forbidden:

```

1 int x, y, z, dummy;
2 int r0 = 42;
3 int r1 = 43;
4
5 void thread0(void)
6 {
7     WRITE_ONCE(x, 1);
8     dummy = xchg(&z, 1);
9     WRITE_ONCE(y, 1, memory_order_relaxed);

```

```
10 }
11
12 void thread1(void)
13 {
14     r0 = smp_load_acquire(&y);
15     r1 = READ_ONCE(x);
16 }
```

In contrast, the closest C11 analog (from David Majnemer and analyzed by Chandler Carruth) does not prohibit this outcome:

```
1 atomic_int x, y, z;
2 int r0 = 42;
3 int r1 = 43;
4
5 void thread0(void)
6 {
7     atomic_store_explicit(x, 1, memory_order_relaxed);
8     atomic_store_explicit(z, 1, memory_order_seq_cst);
9     atomic_store_explicit(y, 1, memory_order_relaxed);
10 }
11
12 void thread1(void)
13 {
14     r0 = atomic_load_explicit(y, memory_order_acquire);
15     r1 = atomic_load_explicit(x, memory_order_relaxed);
16 }
```

Note that this category includes non-value-returning atomics enclosed within `smp_mb__before_atomic()/smp_mb__after_atomic()` pairs. For example, the Linux-kernel variant of `thread0()` could be written as follows with the same outcome, assuming that `z` is declared as `atomic_t` instead of `int`:

```
1 void thread0(void)
2 {
3     WRITE_ONCE(x, 1);
4     smp_mb__before_atomic();
5     atomic_inc(&z);
6     smp_mb__after_atomic();
7     WRITE_ONCE(y, 1, memory_order_relaxed);
8 }
```

Control Dependencies

The Linux kernel provides control dependencies and C11 does not, so `READ_ONCE()` must either retain its current implementation or must be promoted to acquire.

So You Want Your Arch To Use C11 Atomics...

So suppose that you want your Linux-kernel architecture to use C11 atomics. How should you go about it? This section looks at three scenarios: (1) A new architecture, (2) Partial conversion of an existing architecture, and (3) Full conversion of an existing architecture. Each of these is covered by one of the following sections.

New Architecture

The potential advantages of using the C11 memory model for a new architecture include:

- 1. Delegating implementation of atomic primitives to the compiler.
- 2. If multiple architectures take this approach, a reduction in the amount of architecture-specific code.
- 3. The compiler can undertake more optimizations. It is left to the reader to decide whether this would be an advantage or a disadvantage.

`READ_ONCE()`, `WRITE_ONCE()`, and `ACCESS_ONCE()` should continue to use the existing definitions in order to avoid the C11-mandated use of locking for oversized objects.

Memory barriers could be implemented in terms of the C11 `atomic_signal_fence()` and `atomic_thread_fence()` functions as follows:

Linux Operation	C11 Implementation
<code>barrier()</code>	<code>atomic_signal_fence(memory_order_seq_cst())</code> (if safe)
<code>barrier()</code>	<code>__asm__ __volatile__("" : : "memory")</code> (otherwise)

<code>smp_mb()</code>	<code>atomic_thread_fence(memory_order_seq_cst)</code> (if safe)
<code>smp_mb()</code>	Inline assembly otherwise
<code>smp_rmb()</code>	<code>atomic_thread_fence(memory_order_acq_rel())</code> (if safe and efficient)
<code>smp_rmb()</code>	Inline assembly otherwise (otherwise)
<code>smp_wmb()</code>	<code>atomic_thread_fence(memory_order_acq_rel())</code> (if safe and efficient)
<code>smp_wmb()</code>	Inline assembly otherwise (otherwise)
<code>smp_read_barrier_depends()</code>	As in the Linux kernel
<code>smp_mb__after_atomic()</code> and <code>smp_mb__before_atomic()</code>	Depends on implementation of non-value-returning read-modify-write operations
<code>smp_mb__after_unlock_lock()</code>	Depends on implementation of locking primitives

The Linux kernel's locking primitives will likely need to remain as hard-coded assembly for some time to come, particularly for the locking primitives that interact with irq or bottom-half environments. Over time, it might well prove that the compiler can generate “good enough” locking primitives, but careful analysis and inspection should be used to make that determination.

The `atomic_t` and `atomic_long_t` types could be implemented as volatile `atomic int` and `long`. However, this would require inspecting the code that the compiler emits to ensure that the value-returning atomic read-modify-write primitives provide full ordering both before and after, as required for the Linux kernel. Because the C11 compiler might perform optimizations that violate the full-ordering requirement (optimizations based on the assumption of data-race freedom being but one example), it would be wise to add `barrier()` directives at the beginnings and ends of the definitions of the value-returning atomic read-modify-write primitives. This prevents the compiler from carrying out any code-motion optimizations across the `barrier()` directive. In addition, because the C11 compiler can in many cases optimize away atomic operations whose results are not used, and their ordering properties with them, it is wise to place `atomic_thread_fence(memory_order_seq_cst)` before and after C11 atomics that are used to implement Linux-kernel value-returning read-modify-write atomics. Note that this assumes that the implementation in question provides `atomic_thread_fence(memory_order_seq_cst)` ordering properties compatible with `smp_mb()`.

The generic atomic operations might be implemented by casting to volatile atomic objects, and, failing that, inline assembly as is currently used in the Linux kernel.

Until such time as the C11 memory model implements control dependencies, the Linux kernel must implement them. Similarly, RCU must currently also be implemented by the Linux kernel rather than the compiler. That said, TSO machines (for example, x86 and the mainframe) can use volatile `memory_order_consume` loads to implement the `rcu_dereference()` family of primitives without incurring performance penalties.

Partial Conversion of Existing Architecture

In some sense, a new architecture has less to lose by letting the compiler have a go at implementing atomic operations and memory barriers. In contrast, an existing architecture likely already has well-tested high-performance primitives implemented with inline assembly. Not only that, existing architectures might need to support older compilers that do not have robust implementations of C11 atomics. Therefore, any change to C11 should be implemented cautiously, if at all. One way of proceeding cautiously is to do a partial conversion, preferably permitting easy fallback to the original inline assembly. The non-value-returning read-modify-write atomics are likely the safest and easiest C11 primitives to start with.

Full Conversion of Existing Architecture

Full conversion of an existing architecture to C11 requires even more bravery, to say nothing of more complete validation of the relevant C11 functions. For example, it would be wise to provide a Kconfig option selecting between the existing inline assembly and the C11 atomics. This would permit continued use of old compilers where needed, and also allow users to decide when they are ready to trust C11.

Summary

This document makes a first attempt to present a formalizable model of the Linux kernel memory model, including variable access, memory barriers, locking operations, atomic operations, control dependencies, and RCU grace-period relationships. The general approach is to reduce the kernel's memory model to some aspect of memory models that have already been formalized, in particular to those of C11, C++11, ARM, and PowerPC.